

Task 1

To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test 4 Signal Events

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void handler(int sig)
{
    if (sig == SIGINT)
        printf("\nSIGINT received\n");

    else if (sig == SIGTERM)
        printf("\nSIGTERM received\n");

    else if (sig == SIGTSTP)
        printf("\nSIGTSTP received\n");

    else if (sig == SIGUSR1)
        printf("\nSIGUSR1 received\n");
}

int main()
{
    signal(SIGINT, handler);
    signal(SIGTERM, handler);
    signal(SIGTSTP, handler);
    signal(SIGUSR1, handler);

    printf("Process PID: %d\n", getpid());
    printf("Testing 4 signals...\n");

    while (1)
    {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}
```

Output:

```
geya@LenovoL480:~/OS/Skill/Week 10$ ./task1
Process PID: 1461
Testing 4 signals...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
```

Task 2

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt 4 Handling

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child_pid = -1;

void handle_sigint(int sig)
{
    printf("\nShell received SIGINT\n");

    if (child_pid > 0)
    {
        printf("Sending SIGINT to foreground job...\n");
        kill(child_pid, SIGINT);
    }
}

int main()
{
    int status;

    signal(SIGINT, handle_sigint);

    printf("Mini shell PID: %d\n", getpid());
```

```

child_pid = fork();

if (child_pid == 0)
{
    printf("Foreground job started. PID = %d\n", getpid());

    while (1)
    {
        printf("Foreground job running...\n");
        sleep(2);
    }
}
else
{
    printf("Press Ctrl+C to interrupt the foreground job.\n");

    waitpid(child_pid, &status, 0);

    if (WIFSIGNALED(status))
    {
        printf("Foreground job terminated by signal %d\n",
               WTERMSIG(status));
    }

    child_pid = -1;

```

```

        printf("Job state: Done\n");
    }

    return 0;
}

```

Output:

```
geya@LenovoL480:~/OS/Skill/Week 10$ ./task2
Mini shell PID: 1496
Press Ctrl+C to interrupt the foreground job.
Foreground job started. PID = 1497
Foreground job running...
Foreground job running...
Foreground job running...
^C

Shell received SIGINT
Shell received SIGINT
Sending SIGINT to foreground job...
Foreground job running...

Shell received SIGINT
Foreground job running...
Foreground job running...
^_Quit ./task2
geya@LenovoL480:~/OS/Skill/Week 10$ |
```

Task 3

To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

struct Job
{
    int id;
    pid_t pid;
    char state[20];
};

struct Job job;

void handle_sigstop(int sig)
{
    printf("\nSIGSTOP received\n");

    if (job.pid > 0)
    {
        kill(job.pid, SIGSTOP);
        printf("Job %d suspended\n", job.id);
    }
}

int main()
```

```
{
    int status;

    job.id = 1;

    job.pid = fork();

    if (job.pid == 0)
    {
        while (1)
        {
            printf("Child job running...\n");
            sleep(2);
        }
    }
    else
    {
        sprintf(job.state, "Running");

        signal(SIGTSTP, handle_sigtstp);

        printf("Job ID: %d\n", job.id);
        printf("PID: %d\n", job.pid);
        printf("State: %s\n", job.state);

        sleep(5);
    }
}
```

```
printf("\nStopping job...\n");  
kill(job.pid, SIGSTOP);  
waitpid(job.pid, &status, WUNTRACED);  
if (WIFSTOPPED(status))  
{  
    sprintf(job.state, "Stopped");  
    printf("Job state: %s\n", job.state);  
}  
sleep(3);  
printf("\nResuming job...\n");  
kill(job.pid, SIGCONT);  
sprintf(job.state, "Running");  
printf("Job state: %s\n", job.state);  
sleep(5);  
printf("\nTerminating job...\n");
```

```
kill(job.pid, SIGTERM);  
waitpid(job.pid, &status, 0);  
sprintf(job.state, "Done");  
printf("Job state: %s\n", job.state);  
}  
return 0;  
}
```

Output:

```
geya@LenovoL480:~/OS/Skill/Week 10$ ./task3
Job ID: 1
PID: 1571
State: Running
Child job running...
Child job running...
Child job running...

Stopping job...
Job state: Stopped

Resuming job...
Job state: Running
Child job running...
Child job running...
Child job running...

Terminating job...
Job state: Done
geya@LenovoL480:~/OS/Skill/Week 10$ |
```

Task 4

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid == 0)
    {
        // Child creates its own process group
        setpgid(0, 0);

        printf("Child PID: %d\n", getpid());
        printf("Child PGID: %d\n", getpgrp());

        while (1)
        {
            printf("Child process group running...\n");
            sleep(2);
        }
    }
}
```

```

else
{
    // Put child into its own process group
    setpgid(pid, pid);

    printf("Parent PID: %d\n", getpid());
    printf("Parent PGID: %d\n", getpgrp());
    printf("Child PGID: %d\n", getpgid(pid));

    sleep(2);

    printf("\nGiving terminal control to child group...\n");

    tcsetpgrp(STDIN_FILENO, getpgid(pid));

    sleep(5);

    printf("\nSending SIGTERM to child group...\n");

    kill(-getpgid(pid), SIGTERM);

    waitpid(pid, &status, 0);

    printf("\nRestoring terminal control to parent...\n");

    tcsetpgrp(STDIN_FILENO, getpgrp());

    printf("Terminal control restored.\n");
}

return 0;
}

```

Output:

```
geya@LenovoL480:~/OS/Skill/Week 10$ ./task4
Parent PID: 1609
Parent PGID: 1609
Child PGID: 1610
Child PID: 1610
Child PGID: 1610
Child process group running...

Giving terminal control to child group...
Child process group running...
Child process group running...
Child process group running...

Sending SIGTERM to child group...

Restoring terminal control to parent...

[2]+  Stopped                  ./task4
geya@LenovoL480:~/OS/Skill/Week 10$ |
```